LLMs for Verification-Guided Specification Repair

Dimitri Copley, Sajjad Shaffaf, and Farnoosh Afshinrad
{dimitrivalentin.copley, sajjad.shaffaf, farnoosh.afshinrad}@mail.polimi.it
Department of Electronics, Information and Bioengineering
Politecnico di Milano
Milan, Italy

***Abstract*—The increasing complexity of software-intensive systems motivates the development of accessible methods for specifying multi-agent missions. Domain-Specific Languages (DSLs) provide expressive abstractions for describing such behaviors and enable the derivation of formally verifiable models. This work investigates the use of Large Language Models (LLMs) as a component of a broader framework for automated DSL specification repair. We propose a repair pipeline in which syntactically incorrect DSL code is iteratively corrected through an automated feedback loop combining a dedicated compiler and LLM-based generation. Compiler diagnostics are incorporated into prompt refinements that guide the model toward syntactically valid specifications. The approach was implemented for the LIRAs DSL and evaluated across multiple multi-agent mission scenarios while varying key parameters such as prompting strategies, few-shot configurations, and model selection. The evaluation focuses exclusively on syntactic correctness with respect to the DSL grammar and validation rules. The results provide insight into the feasibility of LLM-assisted syntax repair and inform the design of future verification-guided specification repair systems.**

***Index Terms*—Large Language Models (LLMs), Domain-Specific Languages (DSLs), Multi-Agent Systems, Formal Verification, Specification Repair.**

I. Introduction

Domain-Specific Languages (DSLs) are an important medium for specifying multi-agent missions because they provide high-level abstractions while preserving the structured representations needed for downstream analysis and verification [1]. In robotics and multi-agent systems, DSLs therefore act as a bridge between informal mission descriptions and formally analyzable models [1]. However, authoring valid DSL specifications remains difficult for non-expert users, especially when the target language imposes strict syntactic rules, restricted vocabularies, and rigid structural constraints [1].

Large language models (LLMs) have recently emerged as a promising mechanism for translating natural-language descriptions into executable or analyzable artefacts [2]. Nevertheless, when the target formalism is a specialised DSL rather than a widely represented programming language, one-shot generation often produces outputs that appear plausible but still violate the grammar and validation rules enforced by the underlying toolchain [1]. This limitation is particularly relevant for LIRAs, where prior work showed that many generated specifications fail to compile and therefore require additional correction before they can be used in practice [1].

To address this limitation, we study automated syntax repair as an intermediate step between natural-language specification and broader verification-guided specification repair [3]. More specifically, we investigate a compiler-in-the-loop setting in which an LLM iteratively revises an invalid LIRAs specification using compiler diagnostics as explicit feedback [3]. The central idea is to use the compiler not only as a final validator, but also as an active source of deterministic repair guidance [3]. In this work, correctness is intentionally restricted to syntax-level validity, namely successful compilation under the LIRAs grammar and validation rules, while semantic correctness of the mission intent is deferred to later verification stages [1], [3]. This framing allows us to isolate the feasibility and limits of syntax repair before addressing richer forms of semantic or property-level feedback [3].

This paper makes two main contributions. First, it instantiates an operational compiler-in-the-loop repair pipeline for the LIRAs DSL, in which compiler feedback is used as a deterministic signal for iterative LLM-guided correction. Second, it provides a controlled empirical evaluation of this repair process by varying model capacity, repair prompt design, and the presence of few-shot repair examples over a fixed benchmark of cached initial artefacts. This contribution is needed because previous generation-focused results identified syntactic invalidity as a major bottleneck, making automated syntax repair a necessary step toward more complete verification-guided specification repair.

II. State of the Art and Related Work

Recent work has investigated the use of LLMs to generate DSL artefacts from natural-language descriptions. [1] In the case of LIRAs, Tagliaferro et al. showed that, although prompting strategies and few-shot examples can improve output quality, many generated artefacts still fail to compile. [1]

Another relevant direction frames specification correction as a verification-guided iterative process in which an external checker provides feedback for successive revisions. [3], [4] Related work has also highlighted the role of prompt structure in deriving software artefacts from informal specifications. [5] Against this background, the present work focuses on syntax-level repair of invalid LIRAs artefacts under compiler feedback, rather than on initial generation alone. [1], [3]

III. Background

Large language models (LLMs) are neural models trained on large-scale text corpora to predict and generate token sequences. At a fundamental level, their behavior is based

on next-token prediction, while many contemporary systems expose this capability through instruction-tuned, message-structured interfaces. In such settings, inputs are organized as ordered messages, typically associated with roles such as system, user, and assistant, allowing the model to condition its output on task instructions, prior interaction context, and external feedback. In the present study, the LLM is accessed through this chat-style interface, as the repair task requires iterative revision of previously generated DSL artefacts in response to compiler diagnostics.

LLMs have demonstrated strong performance in code generation for widely used programming languages, but their reliability decreases when the target language is a domain-specific language (DSL), where outputs must satisfy strict grammatical and validation constraints [2]. In this context, *syntactic checking* determines whether a specification conforms to the formal grammar of the language, whereas *semantic checking* determines whether a syntactically valid specification is also well-formed with respect to language-level consistency rules, such as valid identifier references and declaration-use agreement. This distinction is particularly important for DSLs, since an artefact may appear plausible while still failing compilation or validation. The issue is especially relevant in robotics and multi-agent mission specification, where DSLs provide high-level yet formally analyzable representations.

Prior work on LIRAs showed that few-shot prompting improves generation quality and reduces hallucinations, but important DSL-specific constraints, such as synchronization behavior, remain difficult for LLMs to capture reliably, and many generated artefacts still require manual correction or fail compilation [1]. These results identify compiler-level correctness as a central challenge in LIRAs generation. More broadly, verification-guided specification repair uses external feedback to iteratively refine candidate specifications [3], [4]. In this work, we focus on compiler-guided repair, where correctness is restricted to successful compilation under the LIRAs grammar and validation rules, while semantic correctness of the intended mission is left to later verification stages [3].

The elements of grammar and validations rules of the DSL we considered are:

- **Pattern header** — a unique identifier naming the interaction pattern.
- **Declarations** — definitions of agents, locations, and resources available within the specification.
- **Agent sequences** — ordered and labeled lists of sub-sequences describing each agent's actions.
- **Sub-sequences** — ordered list of conditioned actions, and loop statements.
- **Conclusion block** (optional) — conditions defining successful or failed termination of the pattern.

LIRAs coordinates agents through a synchronization mechanism based on sub-sequence labels. All agents begin execution of the same numbered sub-sequence simultaneously, and the next sub-sequence cannot begin until the previous one has completed for all participating agents. The DSL also uses a restricted action vocabulary and a limited set of conditional constructs. In addition, identifiers appearing in action statements must match previously declared agents, locations, or resources exactly.

When compilation fails, the LIRAs compiler emits structured diagnostics that expose several recurring constraints relevant to automated repair. These include strict case sensitivity for DSL keywords, rejection of actions outside the closed vocabulary, identifier consistency requirements between declarations and usage sites, cascading diagnostics generated by upstream errors, and sensitivity to whitespace and newlines in the parser. Taken together, these properties define a narrow validity space that LLM-generated artefacts often fail to satisfy on the first attempt.

Large language models operate under a bounded context window, which limits how much prior interaction can be taken into account at each step. In iterative settings that accumulate code fragments and diagnostics, naively appending all past turns risks reaching this limit and increases the influence of early, low-quality attempts on later outputs. To address these issues, interaction histories are often managed with windowing schemes. Fixed-history schemes retain all turns within the context budget, whereas sliding-window schemes keep only the most recent portion of the interaction. Sliding windows constrain prompt growth and reduce the impact of outdated or erroneous content, while preserving enough recent context for the model to generate coherent, locally informed updates.

IV. Methodology

This section presents the implemented compiler-in-the-loop pipeline for repairing syntactically invalid LIRAs DSL specifications into compiler-valid programs, as shown in Fig. 1. The process is divided into sequential stages. In Stage 1, an initial DSL candidate is generated from a natural-language mission description. In Stage 2, the candidate is compiled with the LIRAs compiler to assess whether it satisfies the DSL grammar and validation rules. If compilation succeeds, the process terminates with a valid specification. Otherwise, the compiler diagnostics are extracted and passed to the repair stage, where the LLM produces a revised candidate. Stages 2 is repeated until successful compilation is achieved or the repair iteration limit is reached.

The remainder of this section is organized as follows. Section IV-A details the pipeline architecture and iterative repair workflow. Section IV-A3 describes the repair prompt design. Section IV-B introduces the observational error metric used for convergence analysis.

A. Pipeline Architecture

The system is implemented in Python and consists of generation, compilation, and iterative repair stages. A central architectural decision is the use of *two independent chat sessions* with separate system prompts and separate conversation histories: (i) a generation chat and (ii) a repair chat. This separation isolates generation-time context from the repair loop and supports controlled experimentation, avoiding biases due to prior knowledge and increasing the modularity and reusability of the approach.

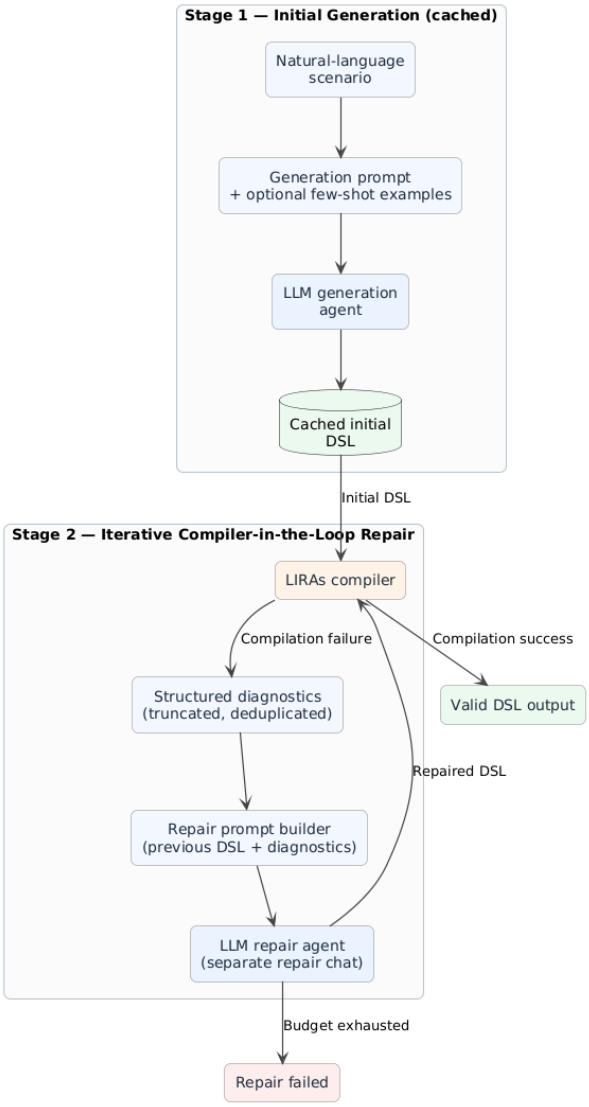

Fig. 1. Compiler-in-the-loop pipeline for syntactic repair of LIRAs specifications. Blue nodes denote LLM-based prompting and repair steps, orange denotes compiler-based validation, green denotes cached or valid DSL artefacts, and red denotes termination after repair-budget exhaustion.

1) Generation chat and caching: For each scenario, an initial DSL artefact is generated from natural language using a system prompt SP1–SP5, temperature $T_{\text{gen}} = 1.0$, and optional *generative few-shot exemplars*. The five prompts vary along two orthogonal axes: *specification modality*—whether the LIrAs language is conveyed through natural-language prose (SP1, SP2), its formal Xtext grammar (SP3, SP4), or a minimal pseudo-code template (SP5)—and *explanatory depth*—whether the prompt includes only the bare specification or supplements it with semantic annotations such as loop behaviour, conditional-statement semantics, and rule-level commentary (SP2 and SP4, respectively). When enabled, exemplars are drawn from a fixed pool of two scenario pairs. Generation is pre-computed and cached: each (scenario $\times$ SP $\times$ shot count) combination is generated once, and the resulting DSL is reused as the identical starting point for all repair configurations. Consequently, only the repair phase varies across the eight experimental cells.

2) Compilation and diagnostic extraction: Each DSL attempt is compiled using a local Java-based (Xtext) CLI tool invoked as `java -jar liras_compiler.jar <file>` with a 60 s timeout. Compilation success is defined as exit code 0 with no error output. Otherwise, the compiler output is parsed into diagnostic feedback for the repair stage.

To stabilise the feedback signal and reduce cascading noise, compiler diagnostics are *truncated before prompting*. Only the first five unique `[ERROR]` lines are retained, with duplicate error lines deduplicated. This focuses the model on likely root causes and improves reproducibility across runs.

3) Repair Prompt Design:

- **System Prompt** — Both repair system prompts (SPR1, SPR2) define the model as an "expert DSL repair agent", expose the initial generated DSL, the most recent failed attempt, and current compiler diagnostics, and enforce a minimal-edit principle: address only the first root error, treating others as potential cascades. They also impose identical output constraints: plain-text LIRAs DSL only, with pattern name on the first line. SPR1 uses a reactive, example-based approach with three concrete error–cause–fix mappings for common generation failures (parenthesis misuse, invalid action verbs, cascading identifier errors), expecting the model to generalize. SPR2 adopts a proactive, rule-based approach with seven prioritized heuristics covering keyword casing, block ordering, action vocabulary, sub-sequence validity, identifier consistency, end-of-file handling, and anti-stagnation delta reasoning. Thus, SPR1 targets high-frequency patterns compactly, while SPR2 provides broader, deductive coverage at the cost of increased prompt length.
- **Few-Shot Learning** — When enabled, three repair few-shot exemplars demonstrate error correction. Each pair shows a malformed DSL and compiler diagnostics paired with the corrected specification, collectively covering case-sensitivity violations, invalid actions, multi-action lines, empty sub-sequences, and conditional misplacement. Responses contain only the corrected DSL, reinforcing output constraints. These exemplars are disjoint from generative few-shots and focus exclusively on transforming invalid to compiler-valid artefacts.

4) Repair chat: loop mechanics: When compilation fails, repair proceeds for at most $k_{\max} = 10$ iterations using a separate repair chat with system prompt SPR1 or SPR2, temperature $T_{\text{rep}} = 0.35$, and optional *repair few-shot exemplars*.

The repair chat is stateful, but uses a *sliding window* of size 3 to limit accumulated context, following the general considerations on context-window management discussed in Section III. At each iteration, the retained history consists of (i) a fixed prefix containing the system prompt and any enabled few-shot messages and (ii) only the most recent three repair turns (six messages total: three user prompts and three model responses). Older turns are pruned prior to constructing the next repair request.

One repair instance = 1 chat using 1 SPR and for each following query Each repair user prompt is structured with explicit sections:

- `REPAIR_TASK`: high-level objective.
- `PREVIOUS_FAILED_ATTEMPT`: full DSL that failed compilation.
- `CURRENT_COMPILER_OUTPUT`: truncated, deduplicated diagnostics.
- `INSTRUCTION`: fix the first error, avoid repeating prior edits, output the full corrected DSL only.

To quantify the context-length savings, consider a representative ten-iteration repair session under configuration C1 (SPR1, no few-shot exemplars). The system prompt contributes approximately 345 tokens per call (SPR2: ∼645), while each user message—comprising a fixed template (∼75 tokens), the previous failed DSL (∼200–600 tokens), and truncated compiler diagnostics (∼150 tokens)—averages 425–825 tokens depending on scenario complexity. Under a full-history policy, the Gemini API re-processes the entire conversation history at each call, causing per-call input to grow from ∼935 tokens (first iteration) to ∼8,600 tokens (ninth iteration), for a cumulative total of approximately 43,000 tokens. The sliding window of $N = 3$ turns caps per-call context at ∼3,800 tokens after the third iteration, reducing the cumulative total to 28,600 tokens—a 33% savings that also limits exposure to early, failed artefacts.

5) Robustness guards and output sanitization: Three guards improve end-to-end reliability:

- *Stagnation detection:* if the model output is byte-identical to the input DSL, the system retries once at a higher temperature (0.7) to break deterministic loops.
- *Empty-response guard:* if the model returns an empty response, the previous DSL is preserved for the next iteration.
- *Output post-processing:* model outputs are sanitized to remove markdown code fences and conversational preamble prior to compilation.

B. Observational Error Metric

In addition to binary success/failure, we compute an *error score* per iteration as a lightweight severity heuristic to track convergence. The score is computed as

$$\texttt{score} = (\texttt{error_lines} \times 10) + (\texttt{warning_lines} \times 2) + \left\lfloor \frac{\min(\texttt{feedback_length}, 2000)}{200} \right\rfloor . \quad (1)$$

where `error_lines` and `warning_lines` count compiler-output lines matching `error` and `warning`, and the final term adds a small capped penalty for long feedback. This metric is used for analysis only and does not affect repair-loop control flow.

V. Experimental Evaluation

This section presents the empirical evaluation of the proposed compiler-in-the-loop repair pipeline. Section V-A describes the experimental design, Section V-B defines the evaluation metrics, and Section V-C reports the results.

TABLE I
Experimental Factors and Levels

Factor	Levels
Model capacity	Gemini 2.5 Flash, Gemini 3 Pro Preview
Repair system prompt	SPR1, SPR2
Few-shot repair examples	Absent (0-shot), Present (3-shots)

A. Experimental Design

To evaluate the effectiveness of the proposed syntax-repair approach, we conducted a 2^3 factorial experiment analyzing the influence of three independent variables on repair performance. The selected factors correspond to model capacity, repair prompt design, and the presence of in-context repair demonstrations (few-shots).

Specifically, model capacity refers to the intrinsic reasoning and code-generation capabilities of the underlying LLM, which we operationalize by comparing a lightweight model (Gemini 2.5 Flash) against a highly capable frontier model (Gemini 3 Pro Preview). Repair prompt design determines the structure and explicitness of the instructions provided to the model (minimal principles versus explicit heuristics). Finally, in-context repair demonstrations consist of supplying the model with concrete examples of successful syntax corrections directly within the repair prompt.

The combination of these factors yields eight experimental configurations. To ensure a fair and comparable evaluation across all tests, the initial DSL artefacts are generated once and cached. Each configuration is evaluated on the benchmark inherited from the prior generation study [1] and consists of:

- four natural-language robotic scenario specifications;
- five generative system prompts (SP1–SP5).

To ensure that differences in performance arise solely from the experimental factors, all other parameters are held constant across runs:

- maximum repair iterations: $k_{\max} = 10$,
- repair temperature: $T_{\text{rep}} = 0.35$,
- generation temperature: $T_{\text{gen}} = 1.0$,
- compiler timeout: 60 s.

Each run corresponds to applying a single repair configuration to a fixed cached initial DSL artefact associated with one scenario and one generative system prompt. This definition ensures that comparisons across configurations reflect differences in the repair settings rather than variability in the initial generation.

B. Evaluation Metrics

The evaluation focuses on four outcome measures chosen to capture complementary aspects of repair performance.

First, *success rate* measures end-to-end repair reliability and is defined as the fraction of runs that reach a compilable DSL specification within the repair budget $k_{\max} = 10$.

Second, for successful runs, we report the *mean number of repair iterations to convergence*. This metric captures repair efficiency, that is, how quickly a configuration reaches a compiler-valid specification once repair succeeds.

Third, we report the *variance of repair iterations* among successful runs. This metric provides a measure of convergence stability: low variance indicates consistent behavior across runs, whereas high variance suggests sensitivity to scenario difficulty or interaction effects among the experimental factors.

Fourth, for unsuccessful runs, we analyze the *distribution of compiler diagnostics* that remain after the final repair iteration. This analysis is used to identify which categories of errors persist despite iterative repair and therefore to characterize the structural limits of the current approach.

In addition to these outcome measures, we use the observational error score introduced in Section IV-B as a descriptive indicator of error severity across iterations. This score is used only for analysis and visualization; it does not affect the control flow of the repair loop.

C. Results

The evaluation encompassed 480 independent runs and executed 3 016 total repair iterations across 8 configurations, 4 mission scenarios, and 5 generative system prompts. Overall, the pipeline achieved a 49.8% success rate, producing compiler-valid specifications in 239 runs; in the remaining 241 runs, it did not reach a compilable state within the predefined iteration limit.

Table II summarizes the performance of all eight configurations in terms of success rate, mean iterations to convergence, and variance of iterations among successful runs.

In Table II, "Mean It." denotes the mean number of repair iterations among successful runs for that configuration, and "Var. It." denotes the corresponding variance. G. 2.5 F. stands for Gemini 2.5 Flash, and G. 3 Pro stands for Gemini 3 Pro.

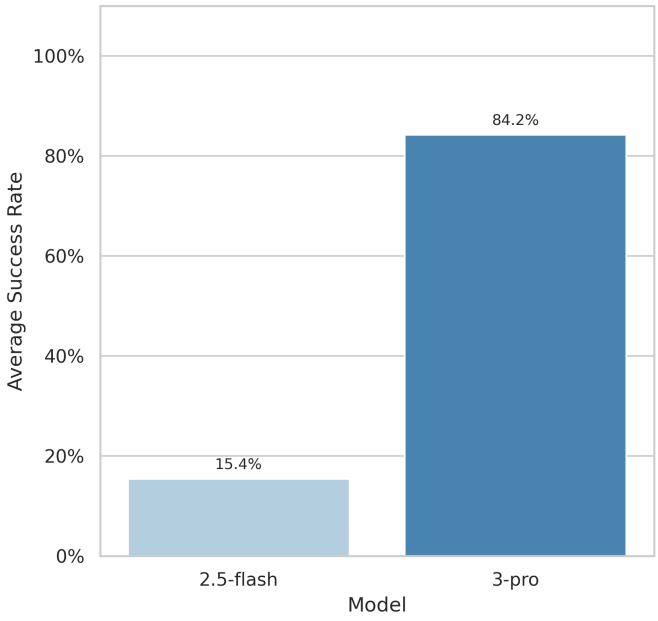

Fig. 2. Main effect of model capacity on repair success rate. The Gemini 3 Pro model demonstrates a substantial baseline advantage over the Gemini 2.5 Flash model.

Figure 2 reports the main effect of model capacity on repair success. Across all prompt and few-shot settings, the Pro model achieves higher success rates than the Flash model.

Figures 3 and 4 report the effect of few-shot repair demonstrations. In the aggregate, adding three in-context repair examples increases the marginal success rate from 40.8% to 58.8%. At the configuration level, the gain is larger for Pro configurations than for Flash configurations.

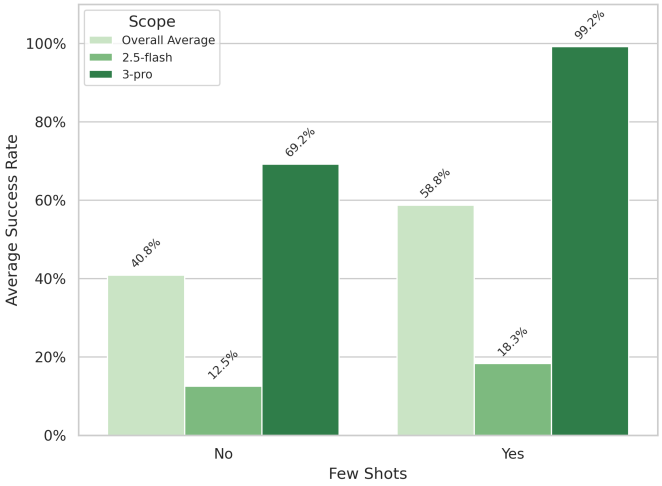

Fig. 3. Effect of few-shot demonstrations on repair success. Providing in-context examples yields measurable improvements, acting as a massive capability amplifier for the Pro model.

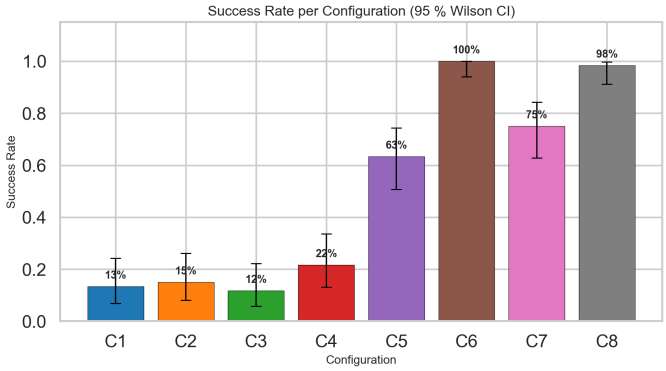

Fig. 4. Success Rate per Configuration with 95% Wilson Confidence Intervals. The inclusion of few-shot examples pushes configurations C6 and C8 to near-perfect reliability, while Flash configurations (C1-C4) remain constrained by high failure rates.

Figures 5 and 6 report the effect of the repair system prompt. The aggregate difference between SPR1 and SPR2 is limited, but the configuration-level results indicate different behavior across model capacities.

Figures 7 and 8 report results across mission scenarios and generative system prompts. These results show that downstream repair performance varies with the quality and structure of the initial generated artefact, particularly in the absence of repair few-shots.

Figures 9 and 10 relate scenario difficulty and initial error severity to final repair outcomes. While some scenarios produce substantially higher initial error scores than others, the

TABLE II
CONFIGURATION SCORECARD AND REPAIR PERFORMANCE

Config	Model	SP	Few-Shot	Rate	Mean It.	Var. It.
C1	G. 2.5 F.	SPR1	No	13.3% (8/60)	2.6	8.0
C2	G. 2.5 F.	SPR1	Yes	15.0% (9/60)	5.3	10.8
C3	G. 2.5 F.	SPR2	No	11.7% (7/60)	5.6	6.3
C4	G. 2.5 F.	SPR2	Yes	21.7% (13/60)	5.0	12.0
C5	G. 3 Pro	SPR1	No	63.3% (38/60)	1.7	0.7
C6	G. 3 Pro	SPR1	Yes	100.0% (60/60)	1.8	2.0
C7	G. 3 Pro	SPR2	No	75.0% (45/60)	1.4	0.5
C8	G. 3 Pro	SPR2	Yes	98.3% (59/60)	1.6	0.8

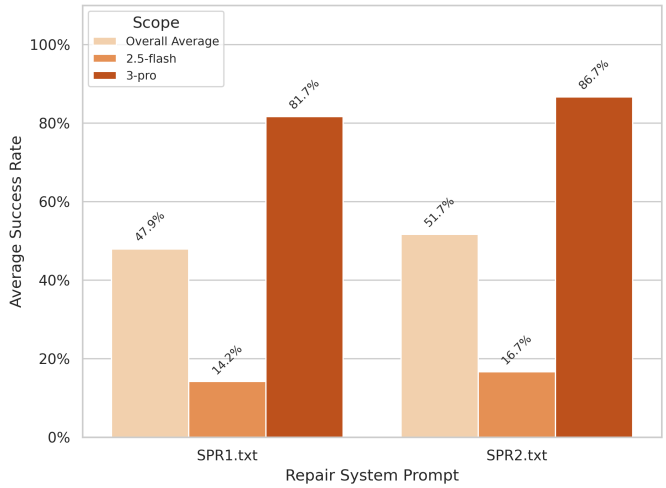

Fig. 5. Impact of repair system prompt tuning on success rates. The performance difference between SPR1 and SPR2 varies significantly depending on the underlying model capacity.

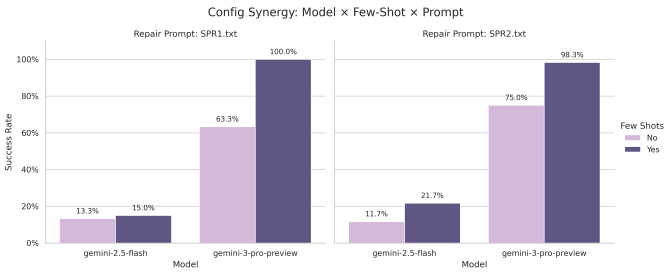

Fig. 6. Configuration Synergy: Model × Few-Shot × Prompt. The side-by-side comparison reveals that SPR2 provides a slight scaffolding benefit to the weaker Flash model, whereas SPR1 pairs optimally with the highly capable Pro model.

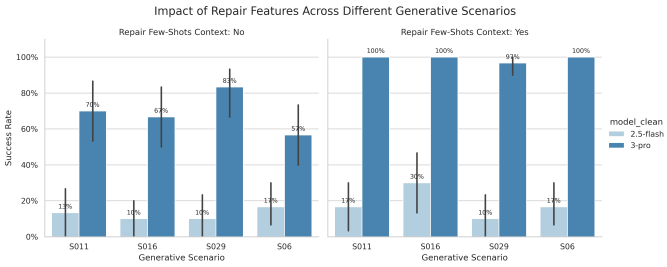

Fig. 7. Impact of Repair Features Across Different Generative Scenarios. Few-shot context normalizes performance for the capable model, overriding inherent scenario complexity.

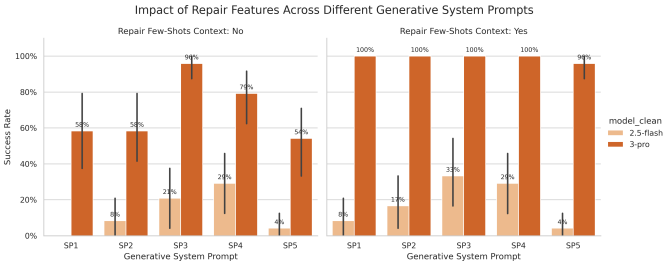

Fig. 8. Impact of Repair Features Across Generative System Prompts. Without few-shot demonstrations, grammar-embedding prompts (SP3) provide a significantly more fixable baseline.

final failure rate varies less across scenarios than the initial error volume alone would suggest.

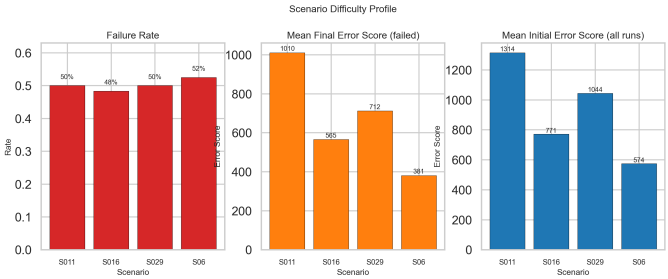

Fig. 9. Scenario Difficulty Profile. Despite large variances in initial and final error volumes, the ultimate pipeline failure rate remains consistent across all evaluated scenarios.

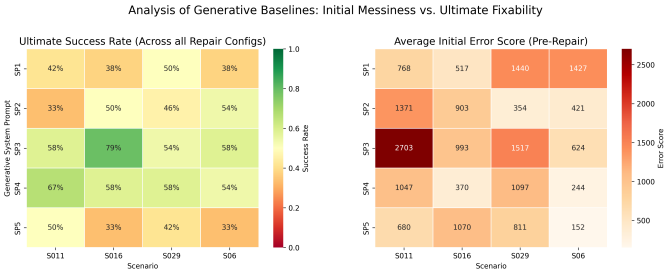

Fig. 10. Analysis of Generative Baselines. The heatmap contrasts the ultimate success rate against the average initial error score, showing that high initial error volume (e.g., SP3 on S011) does not inherently prevent successful repair.

Finally, Figure 11 reports the distribution of compiler error categories before and after repair for the unsuccessful runs. The total number of diagnostics decreases after repair, but some categories remain dominant in the residual error profile.

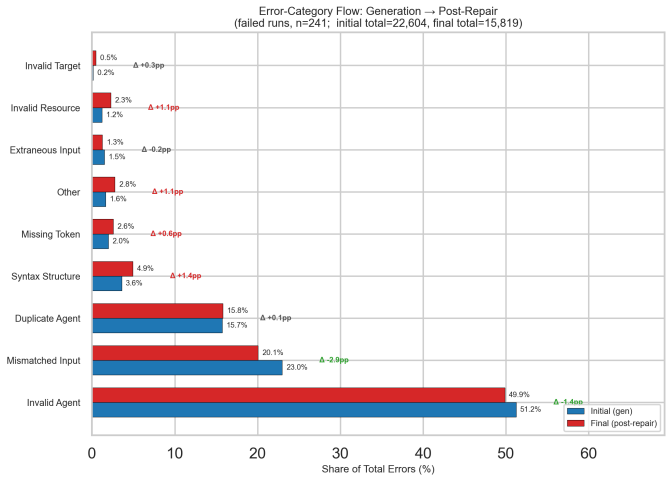

Fig. 11. Error-Category Flow: Generation → Post-Repair. The chart tracks the proportional share of error types across the 241 failed runs. While total error volume decreased by 30%, the share of semantic identifier errors (`Invalid Agent`) remained stubbornly fixed at approximately 50%.

VI. Discussion

This section interprets the empirical results reported in Section V and examines their implications for LLM-guided DSL repair.

A. Effect of Model Capacity

The results show that model capacity is the dominant driver of repair success in this evaluation. As summarized in Table II and Fig. 2, Gemini 3 Pro achieves a marginal success rate more than five times that of Gemini 2.5 Flash (84.2% versus 15.4%).

This gap persists across all prompt and few-shot settings. Even the weakest Pro configuration (C5, zero-shot SPR1) outperforms the strongest Flash configuration (C4, few-shot SPR2) by a factor of nearly three (63.3% versus 21.7%).

The models also exhibit markedly different convergence behavior. Successful Pro configurations typically reach compilation success in fewer than two iterations, whereas Flash configurations often require substantially more attempts and frequently exhaust the repair budget. This pattern suggests that effective DSL repair requires capabilities beyond surface-level pattern matching, including reasoning over document structure and consistency constraints. This asymmetry also carries a practical implication for deployment: investing in a more capable base model appears to yield greater returns than further optimizing prompt design alone. This interpretation is consistent with the view that effective DSL repair requires reasoning over document structure, especially the relationship between declarations and their usage sites, which aligns with the dominant identifier-consistency failure modes observed in the diagnostic analysis.

B. Effect of Prompting Strategies

The two secondary factors considered in this study—repair system prompt and few-shot repair examples—exhibit meaningful but strongly model-dependent effects.

1) Few-shot repair examples: Across both models, adding three in-context repair demonstrations improves the overall marginal success rate from 40.8% to 58.8%. However, the magnitude of this gain depends strongly on the underlying model. For the Pro model, few-shot repair examples substantially improve performance, raising success from 63.3% to 100.0% under SPR1 and from 75.0% to 98.3% under SPR2. Moreover, the 95% Wilson confidence intervals reported in Fig. 4 indicate that these few-shot Pro configurations also exhibit tightly constrained variance, supporting the interpretation that their behavior is not only strong but also highly stable.

In contrast, for the Flash model, the gains from few-shot examples remain limited (C1: 13.3% to C2: 15.0%; C3: 11.7% to C4: 21.7%). A possible explanation is that the weaker effect arises from context-window limitations. However, the prompt-size analysis does not support this hypothesis. The three repair exemplars contribute approximately 831 tokens in total, and even under a worst-case ten-iteration repair trajectory the accumulated prompt remains well below Flash's maximum context length. The exemplars therefore remain fully available throughout the repair process.

The observed asymmetry is thus more consistent with a model-capacity limitation than with a prompt-capacity constraint. Exploiting repair exemplars requires the model to infer an error-to-correction pattern from concrete examples and transfer it to a structurally distinct DSL instance. The Flash model appears to benefit from these demonstrations only weakly, whereas the Pro model derives a substantially larger improvement from the same additional context. Overall, these results suggest that the effectiveness of few-shot repair examples depends less on context-window fit than on the model's ability to generalize from in-context demonstrations.

2) Repair system prompt: The aggregate difference between the minimal principle-based prompt (SPR1 at 47.9%) and the explicit heuristic checklist (SPR2 at 51.7%) is small. However, the configuration-level results reveal a conditional pattern. For the weaker Flash model, SPR2 appears slightly more beneficial when paired with few-shot repair examples, with C4 reaching 21.7% compared with 15.0% for C2 under SPR1. Conversely, for the more capable Pro model, the less prescriptive SPR1 performs equivalently or slightly better, with C6 reaching 100.0% compared with 98.3% for C8.

SPR2 appears slightly more beneficial for the weaker Flash model, particularly when paired with few-shot repair examples. Conversely, the less prescriptive SPR1 performs equivalently or slightly better for the more capable Pro model. This pattern is consistent with the hypothesis that explicit rule enumeration provides scaffolding for weaker models, whereas stronger models can derive the same constraints from higher-level instructions without requiring verbose heuristic guidance.

C. Effect of Generative Baselines

Although the generative phase is separated from the repair loop, the cached starting artefacts provide useful insight into downstream repairability. When repair few-shots are enabled, the Pro model largely neutralizes differences across scenarios and generative system prompts, achieving near-perfect success across conditions.

In contrast, under zero-shot repair, the quality and structure of the initial generation baseline become a major determinant of repair success. Grammar-aware generative prompts, especially SP3 and SP4, provide substantially more repairable starting points than prose-based or template-based alternatives. In particular, the high zero-shot repair success obtained from SP3 artefacts suggests that explicitly embedding DSL grammar constraints during generation anchors the output within a more repairable syntactic regime. More concretely, the Pro model achieved a 96% zero-shot repair success rate when starting from SP3 artefacts, which suggests that grammar-embedding prompts place the generated specification within a particularly fixable syntactic corridor.

The scenario-level analysis also indicates that raw initial error volume is not, by itself, the best predictor of repair success. Some high-error starting artefacts remain repairable, whereas others with fewer initial diagnostics are not. This suggests that the type and distribution of errors introduced during generation matter more than the absolute number of compiler messages. A concrete example is scenario S011, which exhibits substantially higher mean initial error scores than the other scenarios, yet does not produce a proportionally higher ultimate failure rate. More specifically, artefacts generated with SP3 for S011 reach the highest average pre-repair error score (2703) while still achieving a 58% ultimate success rate, reinforcing the claim that error type is more predictive of repairability than raw initial error volume.

D. Error Persistence and Structural Limits

A central limitation of the current approach is the persistence of semantic consistency errors. Here, "semantic" refers to compiler-detected global consistency violations within the DSL, such as invalid identifier references, rather than semantic correctness of mission intent.

The analysis of unsuccessful runs (Fig 11) shows that the repair loop reduces the absolute number of diagnostics by approximately 30%, from 22,604 initial errors to 15,819 post-repair errors. However, the proportional composition of the remaining errors changes much less. In particular, Invalid Agent errors account for roughly half of all diagnostics both before repair (51.2%) and after repair (49.9%).

This pattern indicates that the current local-edit repair strategy is effective at resolving structural and token-level syntax problems, but less effective at enforcing document-wide consistency constraints. Identifier agreement requires global reasoning across declarations and usage sites, whereas the present repair loop is guided by localized compiler feedback and minimal edits.

Moreover, the post-repair increase in some categories, such as Invalid Resource and Invalid Target, suggests that local corrections may inadvertently introduce new consistency violations elsewhere in the document. This motivates future work on constraint-aware decoding, rule-based post-processing, and other mechanisms that can enforce global consistency more explicitly.

E. Threats to Validity

Several design choices limit the scope of the present evaluation.

First, the use of cached initial artefacts ensures that all repair configurations are applied to identical inputs, thereby isolating repair effects from generation variance. However, this choice prevents analysis of whether repair-aware priming would alter the distribution of initial generations.

Second, the repair loop uses a sliding window of three turns in order to limit prompt growth and reduce exposure to earlier failed artefacts. Although this choice is motivated by the context-management considerations discussed in Section III, the optimal window size may depend on model capacity and remains an open question.

Third, 18 runs (3.8%) terminated because of API-level failures, such as rate limiting or safety triggers, rather than pipeline logic. These runs are treated as unsuccessful in the reported results, yielding a conservative estimate of end-to-end reliability.

Finally, the evaluation is conducted on a single DSL (LIRAs) and a single compiler toolchain. Generalization to other DSLs, compilers, and diagnostic formats therefore remains to be established.

VII. Conclusion

This study presented a systematic, cross-comparative evaluation of LLM-guided, compiler-in-the-loop repair for a Domain-Specific Language (DSL), with the goal of identifying which factors most strongly determine whether an automated repair pipeline is reliable and practical. In our setting, the "verification-guided" signal is deterministic compiler feedback: the LIRAs compiler provides grammar-level correctness checks that guide iterative repair until the specification compiles or an iteration budget is reached. Verifying mission *meaning* via translation to a formal model and subsequent property checking is outside the present scope. Importantly, the pipeline also automates and records reproducible metrics—including success/failure, iterations to converge, and categorized compiler diagnostics—enabling controlled comparisons across configurations.

Across 480 runs and more than 3,000 repair iterations, four findings consistently emerged. First, model capacity is the dominant driver of repair success, outweighing prompt variations. Second, few-shot repair demonstrations act mainly as an amplifier for already capable models, while offering limited benefit when base capability is low. Third, the initial generation baseline strongly influences zero-shot repairability; generation prompts that explicitly embed grammar constraints tend to produce outputs that remain within a more repairable syntactic space, even when initial error counts are high. Fourth, global consistency constraints impose a structural ceiling: document-wide identifier consistency is difficult to recover through local iterative edits, so the loop functions well as a syntactic filter but is less effective at enforcing global semantic consistency. These results clarify how compiler-in-the-loop repair can serve as a practical first stage in the broader verification-guided specification framework: stabilizing DSL

syntax to ensure that downstream translation and analysis are feasible.

Future work should extend both automated metrics and feedback signals beyond compilation, incorporating verification outcomes such as property violations and counterexamples to drive semantic repair. Additionally, a wider empirical study is necessary to validate the generalizability of these findings. This expanded evaluation should encompass a broader range of multi-agent mission scenarios, apply the repair pipeline to different Domain-Specific Languages (DSLs) with varying syntactic complexities, and benchmark a wider variety of state-of-the-art LLMs. Methodologically, promising directions include constraint-aware decoding to prevent undeclared identifiers, grammar-guided generation to enforce structural validity, and hybrid pipelines that combine LLM flexibility with deterministic post-processing for constraint classes that are straightforward to formalize.

Acknowledgment

The authors thank Alberto Tagliaferro, Livia Lestingi and Matteo Rossi for their guidance and supervision.

Code Availability

The implementation used in this study is publicly available at https://github.com/Bergenheiser/liras-llm-guided-repair.

References

[1] A. Tagliaferro, L. Lestingi, and M. Rossi, "Preliminary study of dsl code generation for robotics with llms," in *International Conference on Principles and Practice of Multi-Agent Systems (PRIMA / Springer)*, 2024.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[3] A. Tagliaferro, "Harnessing llms for verification-guided specification repair," in *Formal Methods: 26th International Symposium, FM 2024 Doctoral Symposium*. Springer, 2024.

[4] M. Kazemi and S. T. Mousavi Jafarkolaei, "Harnessing llms for verification-guided specification repair," Politecnico di Milano, Project Report, 2024.

[5] A. Tagliaferro, S. Corbo, and B. Guindani, "Leveraging llms to automate software architecture design from informal specifications," in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2025.